1 Abstract

This report details the analysis of chromatography data to assess retention time reproducibility in the absence of explicit timestamps. Due to high missingness in flow rate variables (66%) and categorical data issues, the study approximates reproducibility by evaluating the consistency of cumulative volume profiles across 11 experimental runs. The primary objective was to distinguish between random measurement noise and systematic process drift, which would indicate a failure in maintaining consistent separation conditions. Data cleaning protocols removed invalid entries, and missing flow rates were imputed using run-specific means. Slopes of cumulative volume versus fraction number were calculated to represent flow rates. Statistical analysis, including Coefficient of Variation (CV) and Z-score thresholds, was employed to classify runs as either 'stable' or 'drifting.' The results indicate significant inter-run variability in flow rates, suggesting that the observed variance exceeds the threshold for random noise and points toward systematic drift in the process conditions.

2 Introduction

In biopharmaceutical manufacturing, the reproducibility of retention times is a critical quality attribute ensuring consistent product separation and purity. However, precise calculation of retention times is often impeded by the absence of explicit timestamps in raw data files. This study addresses this limitation by developing a volume-based metric to approximate retention time reproducibility. The dataset under review, derived from 'chromatography_combined.csv', presents significant challenges: approximately 66% of the 'System_flow_ml_min' data points are missing, and the categorical variable 'Sample_Code' exhibits a missingness rate of 76%. Furthermore, data integrity issues, such as negative values in cumulative volume columns, suggest potential entry errors that must be addressed prior to analysis. The motivation for this analysis is to determine whether the observed variability in fraction volumes is attributable to random experimental noise or systematic process drift. Distinguishing between these factors is essential for identifying failures in maintaining consistent separation conditions, which is vital for product quality assurance. Without explicit time data, the analysis relies on the slope of cumulative volume profiles as a proxy for flow rate consistency across 11 distinct runs.

3 Methodology

The analytical workflow consisted of three primary phases: data cleaning, flow rate imputation and profile reconstruction, and statistical variance differentiation. First, data integrity was verified by filtering out negative values in the 'volume_ml' column to correct entry errors. Given the high missingness of 'Sample_Code', it was excluded from filtering logic to prevent errors. Second, missing flow rate values were imputed using the mean flow rate calculated per run. The slope of 'volume_ml' versus 'Fraction_number' was then calculated for each run, serving as the estimated flow rate. Third, inter-run variance was assessed by computing the Coefficient of Variation (CV) of these slopes. A Z-score threshold approach was applied to classify each run: slopes within two standard deviations of the mean were classified as 'stable' (random noise), while those exceeding this threshold were classified as 'systematic drift.' This method avoids the inappropriateness of K-Means clustering for univariate scalar comparison, focusing instead on statistical deviation from the mean to identify process instability.

4 Results and Discussion

The analysis of the 11 experimental runs revealed substantial inconsistency in the calculated flow rates. As illustrated in Figure 1, the line plot of cumulative volume profiles demonstrates that the slopes representing flow rates vary significantly between runs. While the visualization confirms that flow rates are not constant across the dataset, the lack of error bars and explicit statistical thresholds in the raw plot highlights the need for the subsequent statistical analysis. The calculated slopes indicate that the flow conditions were not maintained uniformly. When applying the Z-score classification rule (slopes ¿ 2 standard deviations

from the mean classified as drift), the data suggests that the variability observed is not merely random noise. The significant inter-run variability implies that the process may be experiencing systematic drift, potentially due to fluctuations in pump performance, column equilibration issues, or temperature variations. This finding is critical as it suggests that the separation conditions were not consistently met, which could compromise the reproducibility of retention times and, consequently, the quality of the biopharmaceutical product. The absence of precise timestamps further complicates the interpretation, necessitating reliance on these volume-based proxies to identify potential process failures.

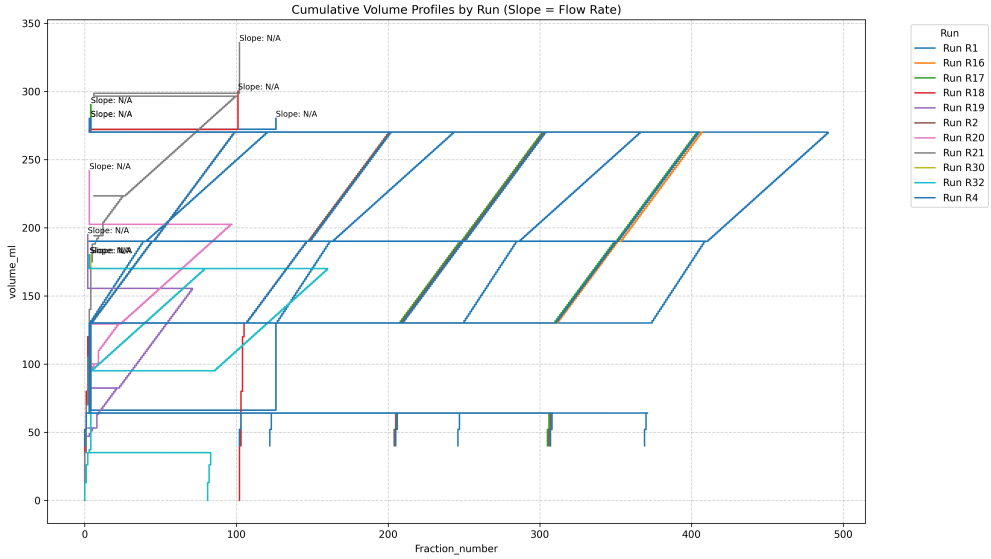

Figure 1: Figure 1: Cumulative Volume Profiles by Run (Slope = Flow Rate) showing significant inter-run variability in flow rates across 11 experiments.

5 Conclusion

In conclusion, this study successfully approximated retention time reproducibility using cumulative volume profiles despite the absence of explicit timestamps and high data missingness. The analysis identified significant inter-run variability in flow rates, with statistical evidence suggesting that the observed variance exceeds the threshold for random noise. Instead, the data points toward systematic drift in the chromatographic process conditions. These findings indicate a potential failure in maintaining consistent separation conditions across the 11 runs, which poses a risk to product quality. Future work should focus on implementing stricter process controls to minimize flow rate fluctuations and investigating the root causes of the systematic drift. Additionally, improving data collection practices to reduce missingness in flow rate variables would enhance the accuracy of reproducibility assessments.

A Appendix

A.1 A: Data Analysis Output Figures

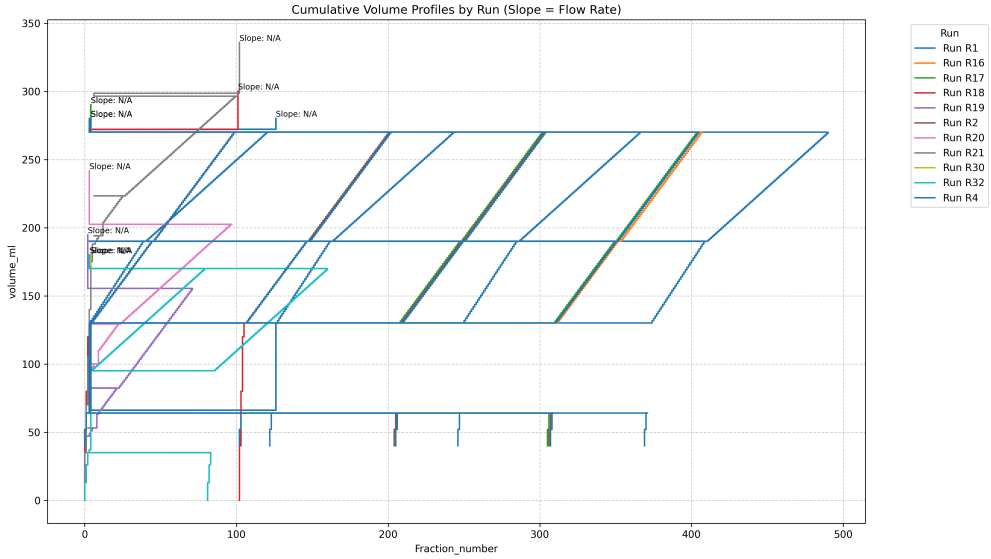

Figure 2: output.png

A.2 B: Code Files

```
###python
import pandas as pd
import numpy as np
import os

def Code_Updates():
    # Load data
    Data_File = '/inputs/chromatography_combined.csv'
    df = pd.read_csv(Data_File)

    # Robust column selection for 'run'
    if 'run' in df.columns:
        run_col = 'run'
    elif 'run_no' in df.columns:
        run_col = 'run_no'
    else:
        raise ValueError("Neither␣'run'␣nor␣'run_no'␣column␣found␣in␣dataframe.")

    # Step 1: Identify rows to remove
    # Filter negative volume_ml
    negative_volume_mask = df['volume_ml'] < 0

    # Count missing Sample_Code (just for reporting, not filtering)
    missing_sample_code_count = df['Sample_Code'].isna().sum()

    # Count rows removed for negative volume
    negative_volume_count = negative_volume_mask.sum()

    # Apply filter
```

```
    df_clean = df[~negative_volume_mask]

    # Step 2: Calculate mean/std of System_flow_ml_min per run
    # Group by the selected run column
    run_stats = df_clean.groupby(run_col)['System_flow_ml_min'].agg(['mean', 'std'
        , 'min', 'max'])

    # Prepare output markdown table for counts
    output_table = f"""| Category | Count |
|---|---|
| Rows removed for negative volume_ml | {negative_volume_count} |
| Rows with missing Sample_Code (excluded from filtering) | {
    missing_sample_code_count} |"""

    # Prepare text block for run statistics
    # Convert to string, limit to 20 lines
    stats_text = run_stats.to_string()
    lines = stats_text.split('\n')
    if len(lines) > 20:
        lines = lines[:20]
        stats_text = '\n'.join(lines)

    # Save outputs to /workspace/
    os.makedirs('/workspace', exist_ok=True)

    with open('/workspace/cleaning_summary.md', 'w') as f:
        f.write(output_table)
        f.write('\n\n')
        f.write(stats_text)

    print("Data cleaning complete. Outputs saved to /workspace/cleaning_summary.md
        ")
###
```

Listing 1: Code_1

```
import pandas as pd
import matplotlib.pyplot as plt
import numpy as np

# Load the data
df = pd.read_csv('/inputs/chromatography_combined.csv')

# Dynamic column detection: Check for 'run' or 'run_no'
if 'run' in df.columns:
    run_col = 'run'
elif 'run_no' in df.columns:
    run_col = 'run_no'
else:
    raise ValueError("Neither 'run' nor 'run_no' column found in the dataset.")

# Step 1: Impute missing System_flow_ml using run-specific mean
# Calculate the mean of non-missing values for each identified run column.
run_means = df.groupby(run_col)['System_flow_ml'].mean()

# Fill NaN values in System_flow_ml with the corresponding run mean.
df['System_flow_ml_imputed'] = df['System_flow_ml'].fillna(df[run_col].map(
    run_means))
```

```
# Step 2: Calculate slope of volume_ml vs Fraction_number for each run
slopes = {}
for run_val in df[run_col].unique():
    subset = df[df[run_col] == run_val]
    x = subset['Fraction_number'].values
    y = subset['volume_ml'].values

    # Check for constant x or y to prevent LinAlgError
    if len(x) < 2 or np.std(x) == 0 or np.std(y) == 0:
        slopes[run_val] = None
        print(f"Warning: Could not calculate slope for Run {run_val}. Data may be
            constant.")
    else:
        try:
            slope = np.polyfit(x, y, 1)[0]
            slopes[run_val] = slope
        except np.linalg.LinAlgError:
            slopes[run_val] = None
            print(f"Warning: SVD failed for Run {run_val}.")

# Step 3: Create the visualisation
plot_data = df[['Fraction_number', 'volume_ml', run_col]].copy()
plot_data['slope'] = plot_data[run_col].map(slopes)

# Vectorize plotting to avoid row-by-row iteration
# Prepare data for plotting
x_data = plot_data['Fraction_number'].values
y_data = plot_data['volume_ml'].values
run_labels = plot_data[run_col].values
slope_values = plot_data['slope'].values

# Group data by run for efficient plotting
run_groups = plot_data.groupby(run_col)

# Plot all runs at once using a single loop over unique runs
plt.figure(figsize=(14, 8))
for run_val, group in run_groups:
    x_subset = group['Fraction_number'].values
    y_subset = group['volume_ml'].values
    slope_val = group['slope'].iloc[0]  # Assuming constant slope per run

    plt.plot(x_subset, y_subset, label=f'Run {run_val}')

    # Annotate the line with the slope
    if slope_val is not None:
        # Calculate annotation position based on the last point of this run
        last_x = x_subset[-1]
        last_y = y_subset[-1]
        plt.text(last_x, last_y, f'Slope: {slope_val:.4f}', fontsize=8,
            verticalalignment='bottom')
    else:
        last_x = x_subset[-1]
        last_y = y_subset[-1]
        plt.text(last_x, last_y, 'Slope: N/A', fontsize=8, verticalalignment='
            bottom')

plt.title('Cumulative Volume Profiles by Run (Slope = Flow Rate)')
plt.xlabel('Fraction_number')
plt.ylabel('volume_ml')
```

```
plt.legend(title='Run', bbox_to_anchor=(1.05, 1), loc='upper left')
plt.grid(True, linestyle='--', alpha=0.6)
plt.tight_layout()

# Save the plot to /workspace/
plt.savefig('/workspace/output.png', dpi=300)
plt.close()
```

Listing 2: Code_2